⚡

Two GPUs in My Laptop. Now They Both Do Real Work.

How I split LLM chat and embedding workloads across NVIDIA + Intel iGPU for **corvia**, local organizational memory for AI agents.

Rust | ONNX Runtime | llama.cpp | OpenVINO | CUDA

github.com/chunzhe10/corvia

THE PROBLEM

Two Workloads.
One GPU.

corvia's inference server runs embeddings (vector search) and LLM chat (reasoning). Both are GPU-hungry. Both were fighting over the same card.

🔍

Embeddings

ONNX Runtime

+

🧠

LLM Chat

llama.cpp

⬇

BOTTLENECK

RTX 3060

6 GB VRAM, constantly swapping

Intel Iris Xe

Completely idle

😴

THE FIX

Split Them.
One GPU Each.

🔍

INTEL IRIS XE

Embeddings

ONNX Runtime
OpenVINO EP
nomic-embed-text-v1.5

gRPC
↔

🧠

NVIDIA RTX 3060

LLM Chat

llama.cpp
CUDA • full offload
llama 3.2 3B Q4

```
# Startup log
cuda=true openvino=true
embedding → device=gpu backend=openvino
chat → device=gpu backend=cuda
```

Neither GPU blocks the other. Embeddings and LLM run in parallel.

RUNTIME DETECTION

Probe. Resolve. Load.
No Config Needed.

🔍 → ⚙ → ⚡ → 🛡

Probe

Check nvidia-smi, /dev/dri render nodes, libopenvino.so

Resolve

Best backend per model type: CUDA → OV → CPU

Load

Each model gets its own hardware backend

Fallback

GPU missing? Use CPU, but log it. No silence.

24	0	1
unit tests covering every backend path	config files needed for GPU detection	binary handles every backend

fallback_used: true. No silent degradation.

THE MESSY PART

"Just Install the Library"

01 Wrong library installed

OpenCL drivers (for Ollama) ≠ OpenVINO runtime (for ONNX)

02 Missing dependency chain

libtbb12 not in Debian Trixie. Pulled 4 .deb files from Ubuntu Noble

03 Wrong library path

APT installs to /usr/lib/. Probe checked /usr/lib/x86_64-linux-gnu/

None of them were "hard".
Used to take an afternoon — Claude handled it in minutes.

ZERO-CONFIG DEVEX

Open Project.
Both GPUs Ready.

HOST
Detect GPUs
init-host.sh
generates override.yml

BUILD
Install Runtimes
OpenVINO + CUDA
+ Ollama + tools

RUN
Models Load
Probe hardware
auto-assign GPUs

NVIDIA passed through
nvidia-smi detected

Intel /dev/dri mounted
render nodes detected

corvia server live
AI coding assistant gets project memory

corvia is organizational memory for AI agents, and the devcontainer dogfoods it. The AI assistant that helps build **corvia** uses **corvia**'s own knowledge base.

THE FULL STACK

What's Actually Running

RTX 3060
6 GB VRAM
CUDA 13.0

Iris Xe
Alder Lake-P GT2
OpenVINO 2025.3

COMPONENT	RUNTIME	GPU
Embeddings	ONNX Runtime + fastembed	**Intel (OpenVINO EP)**
LLM Chat	llama.cpp (GGUF)	**NVIDIA (CUDA)**
Protocol	gRPC • per-model routing	**Fallback-with-signal**

```
# Route each model to its own backend
$ corvia inference reload \
  --model nomic-embed-text-v1.5 --backend openvino

$ corvia inference reload \
  --model llama3.2 --backend cuda
```

Each model gets its own backend. All Rust. All local. No API keys.

TAKEAWAYS

What I Learned This Week

⚡

Your iGPU is free compute.

Split workloads instead of fighting for one GPU.

🔌

ONNX sessions are independent.

One on CUDA, one on OpenVINO, one on CPU. All parallel.

🛠

Detect at startup, not at build time.

One binary. Probe hardware. Pick backends. Fall back gracefully.

Running local inference on heterogeneous hardware?

Curious what setups people have landed on 👇